SDC HACKATHON 2026

Phase 2: Final Project & Prototype Report

Hosted by Xiamen University Malaysia (XMUM) AI Club
Submission Deadline: 1 November 2026

Instructions: Complete every field below and submit as a single PDF document named "SDC2026_TeamName_Final.pdf" together with your pitch deck and a working deployment link. Answers must comply with the word limits stated for each open-ended question. Section 3 requires concrete, verifiable evidence. Unsupported claims will not be credited by judges.

1. Team & Project Overview

Team Name	Kita Akan Lawan
Project / Product Name	StockNow
Final Live Deployment URL	https://stock-now-theta.vercel.app/
Codebase Repository URL	https://github.com/Albrea/Kita-Akan-Lawan-StockNow.git
Name (Leader)	Alfred Jason Gustono
Email & phone (Leader)	alfredgustono0926@gmail.com / +62 821 6132 7646

2. Project Description

[Maximum 250 words]

StockNow is an AI restock planner for small product-based shops, built as a module for the **Orbyte** business hub. Many Malaysian micro and small businesses, which produce 39.5% of the country's GDP [1], still keep stock in notebooks, track sales in spreadsheets, and order from suppliers on WhatsApp, so "what should I buy, how much, and can I afford it?" is answered by guesswork.

StockNow keeps stock, sales, suppliers, the purchase budget and approvals in one place and answers that question. Its forecasting engine predicts each product's demand for the next 30 days, works out how much to buy, fits the plan into the budget, and gives a buy/wait / don't-buy decision with reasons, expected profit and a confidence score. Every number opens in "Show calculation", and an accuracy check tests the forecast on the shop's own past months.

Google Gemini never decides what to buy. It proposes how StockNow's forecast is tuned for this business, from derived sales statistics; StockNow checks and backtests it, and nothing changes until an owner approves. Gemini also writes the plan summary and supplier WhatsApp messages from the engine's own figures.

People stay in control. Staff send plans, an Owner or Manager approves item by item, approved items become one order per supplier, and every action is logged. Paid customer orders from Orbyte Payment (simulated in the demo) automatically become stock-outs, and anything unmatched goes to a person. The app works on phones or desktops in English, Bahasa Melayu, Tamil, and Chinese.

3. Upgraded Product & Technical Execution

3.1 Final System Architecture

Provide a detailed description (and, if possible, an attached technical diagram) of your fully built system. Explain how your application integrates GenAI APIs, database structures, and backend/frontend frameworks into one unified solution.

StockNow is one Next.js 15 application, written in JavaScript and deployed on **Vercel**. Data lives in Supabase PostgreSQL. Language generation and settings proposals are provided by the **Google Gemini API**. Figure 1 shows how the parts connect.

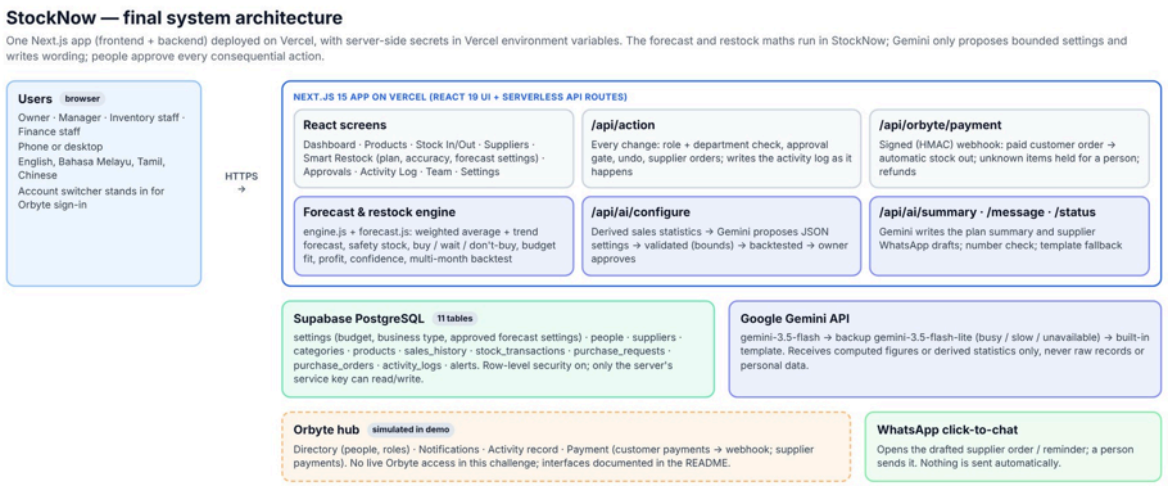


Figure 3.1.1. StockNow final architecture (also in the repository README).

Frontend: Responsive React screens for desktop and phone: Dashboard, Products, Stock In, Stock Out, Suppliers, AI Restock (restock plan, accuracy check, forecast settings), Approvals, Activity Log, Team & access and Settings. A built-in dictionary provides English, Bahasa Melayu, Tamil and Chinese; Language is the first tab in Settings so a non-English reader can find it.

Backend: /api/state loads the shop. Every change goes through /api/action, which runs one function (applyAction) that checks the user's role on the server, validates the input, applies the change and writes the activity-log entry in the same save. The approval gate therefore cannot be bypassed from the browser. Writes are serialised by a lock and saved to Supabase as a diff.

Forecast and restock engine. For each product: forecast = w × weighted average of the last 3 months (3:2:1) + (1 − w) × linear trend, safety stock, quantity needed = demand + safety and stock, already on order. Skip rules (overstock, thin margin, falling demand), a priority score fills the budget, expected profit and ROI. A confidence score from how steady sales are and a rolling backtest that scores the forecast against a simple "average of the last 3 months" baseline. Its six tunable numbers (w, safety %, cautious %, sell window, overstock days, thin-margin %) are stored as data with fixed safe ranges.

GenAI integration:

- **Adaptive forecast settings** (/api/ai/configure): StockNow computes derived statistics (average sales, variability, trend, last month vs earlier, margin, and the measured backtest error of every allowed blend). Gemini returns JSON settings in JSON mode. StockNow rejects unknown names, out-of-range values or weights that do not add to 1, backtests the proposal against the current settings, and shows both errors. Nothing changes until an Owner or Manager presses Approve; the server validates and backtests again before saving.
- **Wording** (/api/ai/summary, /api/ai/message): Gemini turns computed figures into a plan summary and supplier WhatsApp messages. A number check discards any reply containing a number the engine did not produce.
- **Reliability**: Gemini 3.5 flash, if cannot then Gemini 3.5 flash lite. If its slow (12 s) or unavailable, then a built-in template. Each text is labeled with the model that wrote it, and Settings → AI has a "Test Gemini" button (/api/ai/status).

Who decides what? The split of work is fixed in the code:

Part	What it does
StockNow engine (our code)	Forecast, quantity to buy, buy / wait / don't-buy, budget fit, expected profit, confidence, backtest. Same inputs always give the same answer.
Gemini (AI)	Proposes forecast settings with reasons: writes the plan summary and supplier messages. Every reply is checked (ranges, backtest, number check) before it is shown.
Owner / Manager (person)	Approves new forecast settings and every purchase, item by item. Staff can plan and send, not approve.

Database. Supabase PostgreSQL, 11 tables: settings (budget, business type, approved forecast settings with approver and backtest), people, suppliers, categories, products, sales_history, stock_transactions, purchase_requests, purchase_orders, activity_logs and alerts. Row-level security is on and only the server's service key can read or write. Starting settings for each business type are presets in code, what is saved per shop is its chosen type and its approved settings. One upgrade script (supabase/upgrade-final.sql) adds these three columns and the payment alert type. Without Supabase the app falls back to an in-memory demo store.

Orbyte (assumption interface) and WhatsApp. There is no live Orbyte access in this challenge, so StockNow never calls Orbyte. We designed StockNow's side of the connection instead: /api/orbyte/payment would receive Orbyte's paid-order message, signed with HMAC-SHA256 (it is switched off unless a shared secret is set). Matched items become stock-outs, repeated payment IDs are ignored, refunds put stock back, and unknown items or missing stock are held for a person. In the demo, a "Simulate a customer payment" button plays Orbyte's part. Orbyte Directory, eLeave, notifications and the activity record are simulated the same way; the assumed interfaces are documented in the README. Supplier messages open in WhatsApp through click-to-chat, and a person sends them.

One complete journey: trigger (low-stock alert, monthly plan, or Orbyte payment) → connected data (stock, 6 months of sales, open orders, budget) → decision (engine, with settings tuned by Gemini) → human approval (Owner/Manager, per item) → visible outcome (one order per supplier, WhatsApp draft, delivery received, activity log).

3.2 Implementation & Hackathon Deliverables

Describe what was successfully built, integrated, and deployed during the 48-hour physical sprint. Highlight any specific problem-solving steps your team took to pivot or improve the project from the Phase 1 preliminary design.

[Maximum 300 words]

Development and Integration

Built, integrated, and deployed the Phase 1 prototype. Further enhancements included adaptive forecasting with Gemini, owner approval and backtesting, automatic stock deductions from Orbyte Payment, undo for stock movements, real dates, order highlighting, four-language support, a Gemini test button, backup model and enhancing UI.

Pivots and Problem-Solving

1. **Making AI useful.** Initially, Gemini only reworded results. We upgraded it to propose adjustable forecasting settings. However, our first backtest showed increased error (9.5% to 11.7%). We therefore made backtesting the approval gate and provided Gemini with measured errors. Its revised proposal reduced forecasting error to 5.1% (Appendix B).
2. **Reliability.** Under high demand, gemini-3.8-flash returned no text within 30 seconds, triggering silent template fallbacks. We switched to gemini-3.5-flash, added an automatic backup model, model-identification labels, and a Gemini testing button.
3. **Orbyte integration.** Without direct Orbyte access, we designed a signed payment webhook and simulation feature to automatically deduct stock from paid orders, with exceptions requiring human intervention.
4. **Trust and usability.** Added undo through reversing entries rather than deleting records, highlighted unfinished supplier orders, a continuation bar for receiving stock, and unified loading feedback.

5. **Field feedback.** Interviewed five SMEs at KIPMall Kota Warisan (Appendix A), mostly shop staff. Most reported improved efficiency and ease of use, with over 50% perceived improvement. However, one SME struggled to understand the system. This small sample is insufficient for definitive usability conclusions.

Quality Assurance

Conducted 48 automated tests covering forecasting, budgeting, approvals, undo, payments, and Gemini fallback, alongside end-to-end browser testing on the production build.

4. Environmental & SDG Impact Verification

Validate the real-world impact of your built solution. Show concrete data that validate your environmental or social value proposition, compared against the baseline and KPI defined in previous phase submission.

*The data and evidence you provide in this section will be used to assess **SDG Impact, Technical Execution and Innovation scores.** Vague claims or missing verification will weaken your score across all three categories.*

<table>
<tr><th>Metric Category</th><th>Your Response</th></tr>
<tr><td>Target SDG Metric Achievement
Direct comparison between your Phase 1 baseline and your Phase 2 finalized prototype numbers
e.g., “Achieved a 24.5% reduction in computational execution overhead against the 20% initial target.”</td><td>In Phase 1, our team established three measurable KPIs aligned with UN SDG Targets 8.2 and 9.3. These KPIs were designed to measure improvements in demand forecasting, restock-order preparation, and budget control. The following table compares the original Phase 1 baselines and targets with the results achieved by the Phase 2 prototype.

<table>
<tr><th>KPI</th><th>Phase 1 baseline</th><th>Phase 1 target</th><th>Phase 2 result</th></tr>
<tr><td>1. Forecast error (MAPE)</td><td>18.6% using the three-month-average forecasting rule</td><td>At least 30% lower</td><td>7.8% using adaptive settings, representing a reported 58.3% reduction. Default settings achieved 11.0%, a 41.2% reduction.</td></tr>
<tr><td>2. Time to produce an approved, budget-checked order</td><td>Approximately 6 manual tasks per order</td><td>At least 50% less time</td><td>The automated browser test completed the workflow in 2.4–2.9 seconds across three runs. This measures system execution time, not the retailer's total task-completion time.</td></tr>
</table>
</td></tr>
</table>

Metric Category	Your Response
	3. AI-planned purchases within budget | Budget compliance was not enforced | 100% of planned purchases within budget | 100% compliance: all 60 tested plans remained within budget, and over-budget plans could not be submitted. **KPI 1, Forecast accuracy:** The forecasting evaluation used six synthetic demand scenarios covering eight products and 144 monthly test points. The final system applied a rolling backtest to compare its adaptive forecasting settings against the three-month-average baseline. The results showed lower forecast errors across all six scenarios: mixed (5.1% vs. 19.9%), growing (5.4% vs. 22.0%), slow season (1.7% vs. 26.7%), festive (25.6% vs. 30.4%), viral (6.4% vs. 7.1%), and overstock (2.3% vs. 5.8%). **Additional benefit, Orbyte Payment integration:** For matched paid orders, StockNow can automatically record stock-out transactions without requiring staff to manually enter each product, quantity, and reason. Unmatched items are flagged for manual review rather than being processed automatically. This integration is simulated in the demonstration because the team did not have access to a live Orbyte system. **Limitations:** The evaluation uses synthetic shop data rather than actual retailer sales data, in accordance with the Domain 2 requirements. Furthermore, the adaptive forecasting settings were tuned and evaluated using the same historical synthetic dataset. Therefore, the results should be treated as prototype evaluation results rather than proof of performance in real retail environments. Human approval is still required before proposed forecasting settings and purchases are applied.

Metric Category	Your Response
Verification Evidence Method *How was this data verified?* *e.g., codebase algorithmic profiling logs, hardware power-meter simulation readings, or database transaction counters.*	• **Code and automated tests:** npm test runs 48 tests, including the backtest (backtestMulti), the budget limit, server-side approval permissions, undo, Orbyte payment duplicates and refunds, and the Gemini fallback chain. • **Reproducible measurement:** a script runs backtestMulti() and blendErrors() over all six scenarios (output in 4.1), and the planner at 10 budgets per scenario (RM 1,000–50,000) to count over-budget plans. • **Timed browser runs:** Playwright clicked through Plan restock with AI → Send to Approval → Approve & create orders on the production build with a live Gemini key (3 runs, 2.4 s, 2.5 s, 2.9 s). • **In-app evidence for judges:** AI Restock → Accuracy check and Forecast settings show these errors live; the Activity Log records each approval with its backtest result (Appendix B).
Socio-Economic / Civic Value *How does your deployed application translate into a scalable, real-world solution capable of being supported by civic or corporate partners?*	Small shops lose sales when best-sellers run out and lose cash when slow stock piles up. In our demo shop StockNow flags **RM 22,700 tied up** in three items it advises not to reorder. StockNow turns that guesswork into an explainable, approved plan in minutes, without accounting skills. • **Inclusive:** four languages, plain-language reason, and an offline mode that keeps stock entry working during outages. • **Partner-ready:** designed to run inside Orbyte, so SMEs already on the hub switch on a module instead of learning a new system. Business associations, SME agencies or banks could offer it to member shops. A lender could see that purchases follow an approved, budget-checked plan. • **Adaptable:** the business type (clothing, food & beverage, electronics, general retail) sets starting settings, and Gemini tunes them from each shop's own sales.
Deployment & Commercial Feasibility	• **Orbyte module / add-on:** plugs into Orbyte's sign-in, roles, eLeave, notifications, activity record and

Metric Category	Your Response
e.g., modeled as an API service plugin designed for easy integration into existing corporate or institutional systems.	Payment. A freemium model could offer stock tracking free with the AI planner as the paid upgrade. • **Reusable building blocks:** the forecast engine is plain JavaScript with no AI dependency. The assumed Orbyte payment interface (/api/orbyte/payment) and the settings route (/api/ai/configure) are documented APIs other systems can call. • **Low running cost:** the maths runs locally (a plan for 2,000 products × 12 months takes about 93 ms). Gemini is called only for a summary, a supplier message or a deliberate settings refresh, and approved settings are saved, not re-requested. • **Portable:** standard Next.js + PostgreSQL. Moves from Replit to Vercel by changing environment settings.

4.1 Mathematical Proof / Code Logic Snippet (Optional, but holds bonus marks)

Paste the specific code section, mathematical formula, simulation log output, or calculation logic that proves the impact claims made above.

This proof is graded across three categories at once: ***Impact and Feasibility*** *(does the math support your claim),* ***Technical Execution*** *(does it reflect what you actually built), as well as* ***Innovation and Creativity*** *(is the approach original).] Do not leave this section generic or copied from Phase 1.*

(a) Forecast, with the blend now a setting (src/lib/engine.js). With the default $w = 0.5$ the results are identical to Phase 1. Tests confirm this.

```
export function forecastFrom(s, cfg = DEFAULT_FORECAST_CONFIG) {
 const n = s.length;
 const wma = (s[n - 1] * 3 + s[n - 2] * 2 + s[n - 3]) / 6;              // recent months count more
 const xm = (n - 1) / 2, ym = s.reduce((a, b) => a + b, 0) / n;        // least-squares trend line
 let nu = 0, de = 0; s.forEach((y, i) => { nu += (i - xm) * (y - ym); de += (i - xm) ** 2; });
 const slope = nu / de, lin = ym + slope * (n - xm);
 const forecast = Math.max(0, Math.round(cfg.wmaWeight * wma + cfg.trendWeight * lin));
 ... }
```

(b) Rolling backtest against the 3-month-average baseline (MAPE, months with 0 sales skipped).

```
for (let k = 3; k < s.length; k++) {                  // every month that has 3 months before it
 const hist = s.slice(0, k);
 const f = forecastFrom(hist, cfg).forecast;          // StockNow
```

```
  const base = Math.round((hist[k-1] + hist[k-2] + hist[k-3]) / 3);   // the manual rule
  pts.push({ actual: s[k], f, base });
}
error = mean(|f − actual| / actual);   baseline = mean(|base − actual| / actual)
```

(c) Guardrail on Gemini's proposal (src/lib/forecast.js): only known settings, inside fixed ranges, weights adding to 1, otherwise the current settings stay.

```
BOUNDS = { wmaWeight: [0.2, 0.8], safetyPct: [0.1, 0.4], cautious: [0.5, 0.9],
           sellWindow: [1, 2], overstockDays: [14, 90], lowMargin: [0.15, 0.35] }
if (!TUNABLE.includes(k)) errors.push(`Unknown setting "${k}".`);
else if (v < lo || v > hi) errors.push(`${k} = ${v} is outside lo–{hi}.`);
values.trendWeight = round2(1 - values.wmaWeight);   // weights always add up to 1
```

(d) Measurement log

```
scenario  | tests | 3-mo avg | last month | default 50/50 | best w | adaptive
mixed   | 24   | 19.9%   |   10.9%          |    9.5%          | 0.2   |   5.1%
growth  | 24   | 22.0%   |   11.6%          |   10.3%          | 0.2   |   5.4%
slow     | 24   | 26.7%   |   12.7%          |    9.2%          | 0.2   |   1.7%
festive   | 24   | 30.4%   |   24.2%          |   26.8%          | 0.2   | 25.6%
viral     | 24   |   7.1%   |7.1%    |    6.9%          | 0.7   |   6.4%
overstock |  24   |   5.8%   |    4.5%          |          3.2%      | 0.3   |   2.3%
ALL 144 tests: 3-mo avg 18.7% | last month 11.8% | default 11.0% | adaptive 7.8%
reduction vs 3-mo avg: default 41.2%, adaptive 58.3%; adaptive vs default 29.1%
budget: 60 plans, 0 over budget
Gemini (live): proposal wmaWeight 0.2 -> backtest 9.5% -> 5.1%; written by gemini-3.5-flash-lite
```

Originality: the language model does not do the arithmetic. It adjusts a transparent, testable formula within safe limits, using measured evidence, and every change is backtested and approved by a person before it affects a purchase.

5. Market Viability & Future Scalability Plan

SDC aims to encourage solutions that transition from innovations into market-ready applications. If given access to corporate funding, industry grants, or partnership opportunities (e.g., through SDC's Rotary network or joint research initiatives at XMUM), how would your team scale this prototype to process real-world data at volume?

[Maximum 250 words]

Market. StockNow targets Malaysia's product-based micro and small businesses (clothing, groceries, food & beverage, hardware), first through Orbyte's existing SME customers, who could switch it on as a module. Pricing could be freemium: free stock tracking, with the AI planner and Orbyte automations as a paid add-on.

Scaling plan with funding and partners:

1. **Pilot:** onboard the KIPMall shops we interviewed. Add CSV and Excel import with column mapping, preview and owner confirmation, so their existing records become sales history. Real data is used only with consent and handled under the PDPA.
2. **Validate:** test forecast accuracy on real, anonymised sales across shop types, and add seasonality (Raya, 11.11, school holidays) and per-product details (shelf life, lead time, minimum order).
3. **Scale:** Connecting to the real Orbyte system. Nightly forecasts in a job queue (the engine already plans 2,000 products × 12 months in about 93 ms). Gemini called only on demand, with approved settings cached.
4. **Tutorial:** a short guided first-use tour in four languages, with short videos and sample data. Each role learns only its own steps, and Orbyte support can onboard many shops at once.

6. Final Submission Checklist

Tick all boxes that apply before submitting.

☒ The application is fully compiled, live, and accessible via the deployment link provided in Section 1.

☒ The GitHub (or equivalent) repository contains a clean, well-documented README with installation instructions.

☒ The GitHub (or equivalent) repository shows the history of incremental code updates made throughout the official live hacking window.

☒ The final pitch deck has been exported and saved as a flattened PDF file.

☒ All fields in this template have been completed and word limits respected.

7. IP & Originality Declaration

7.1 Pre-Existing / Prior-Owned Work

Does this submission include any code, design, research, or other work that any team member created or owned before the start of this hackathon?

☐ Yes ☒ No ☒ No change from Phase 1 declaration

If yes, briefly describe what was pre-existing and which team member(s) owned it:

7.2 Third-Party Code, Datasets, or Models

Does your project use any third-party code libraries, open-source frameworks, public/private datasets, or pre-trained AI models that your team did not create?

☒ Yes ☐ No ☐ No change from Phase 1 declaration

If yes, list them below (open-source use is permitted provided it is declared and properly licensed/attributed):

Name	Type (code/dataset/model)	License
Next.js 15.5.28	Code (web framework)	MIT
React / React DOM 19.2.0	Code (UI library)	MIT
lucide-react 0.555.0	Code (icon library)	ISC
@supabase/supabase-js 2.117.0	Code (database client)	MIT
Vitest 3.2.7; Playwright (testing only)	Code (test tools)	MIT; Apache-2.0
Inter, Noto Sans SC, Noto Sans Tamil (Google Fonts)	Code (fonts)	SIL Open Font License 1.1
Google Gemini 3.5 Flash / 3.5 Flash-Lite (API)	Model (pre-trained, via API)	Google APIs Terms of Service & Gemini API Additional Terms
DOSM MSMEs Performance 2024 [1]	Dataset (public, cited only)	DOSM terms of use

7.3 Employment-Related IP Conflicts

Is any team member currently bound by an employment, internship, or consulting agreement under which their employer could claim ownership of work they create during this hackathon?

☐ Yes ☒ No ☒ No change from Phase 1 declaration

If yes, please describe the nature of the agreement and which member it concerns:

8. Declaration

By submitting this report, the team leader confirms that the deployed application and data presented above are the original work of the team, completed within the official SDC 2026 hackathon period.



Alfred Jason Gustono

31st October 2026

References

[1] Department of Statistics Malaysia (DOSM), "Micro, Small & Medium Enterprises (MSMEs) Performance 2024," DOSM, July 31, 2025. [Online]. Available: https://www.dosm.gov.my/portal-main/release-content/micro-small--medium-enterprises-msmes-performance-2024

[2] United Nations, Department of Economic and Social Affairs, "Goal 8, Target 8.2: productivity through diversification, technological upgrading and innovation," UN DESA. [Online]. Available: https://sdgs.un.org/goals/goal8

[3] United Nations, Department of Economic and Social Affairs, "Goal 9, Target 9.3: access to financial services and integration of small enterprises into value chains and markets," UN DESA. [Online]. Available: https://sdgs.un.org/goals/goal9

[4] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed., OTexts, 2021, ch. 5.8 "Evaluating point forecast accuracy" and ch. 5.10 "Time series cross-validation." [Online]. Available: https://otexts.com/fpp3/

[5] Google, "Gemini API: generate structured output (JSON mode)," Google AI for Developers. [Online]. Available: https://ai.google.dev/gemini-api/docs/structured-output

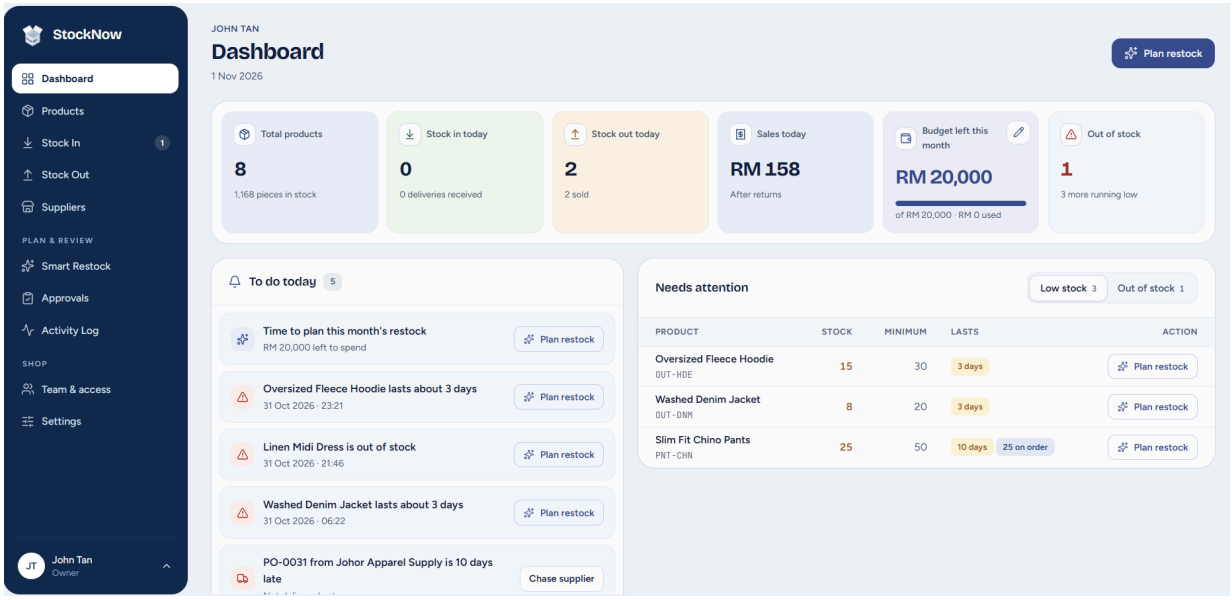
Appendix

Figures A. Proof of Interviewing SMEs in KipMall Kota Warisan Sepang, Selangor, Malaysia

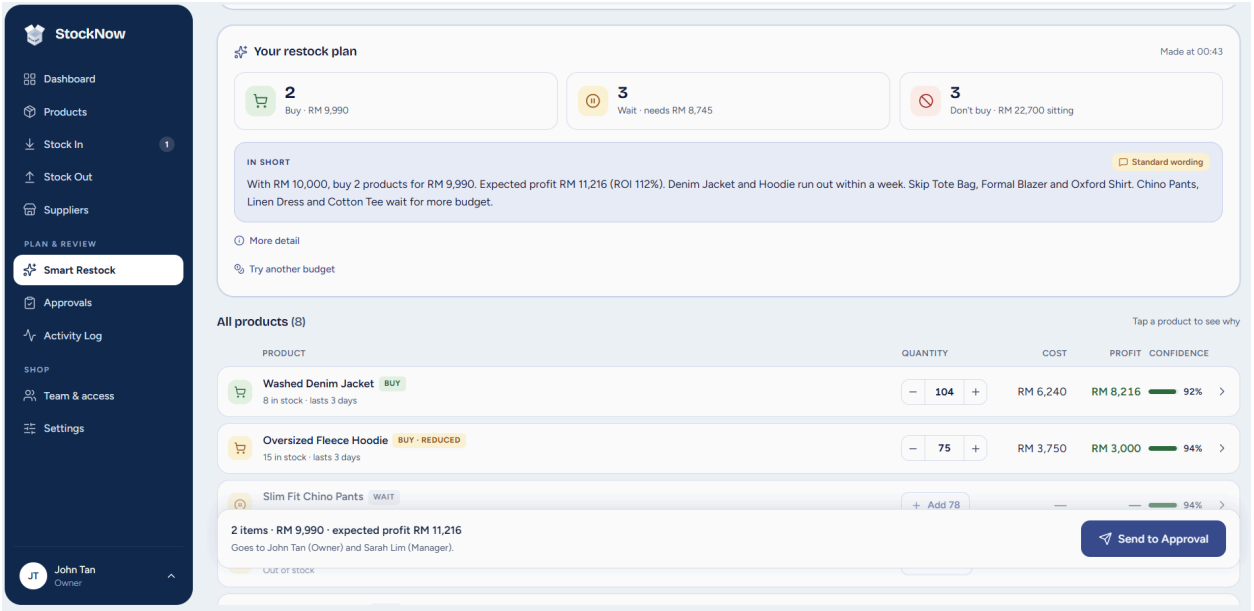


Note: We interviewed five diverse SMEs at KIPMall Kota Warisan: Skechers (footwear), Al-Ikhsan Sports (sporting goods), Switch (Apple electronics), MR. DOLLAR (discount household retail), and an independent clothing boutique. We successfully gathered insights from all five, though we only captured photos with the staff at three locations due to strict in-store photography policies at the other two.

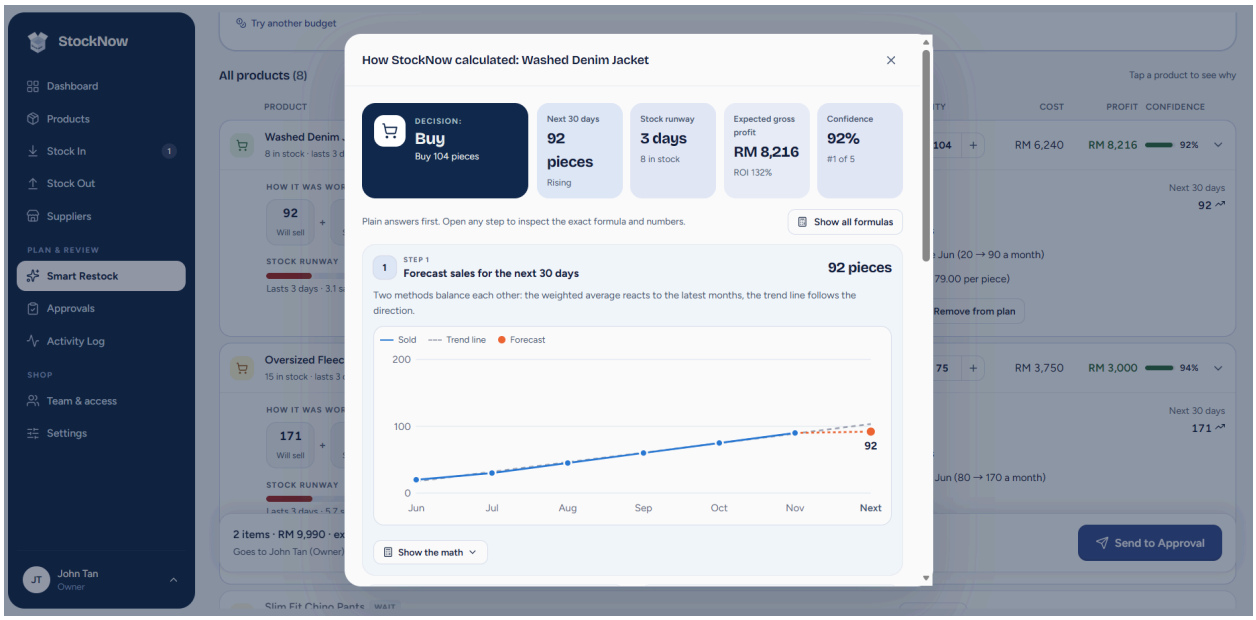
Figures B. The final system



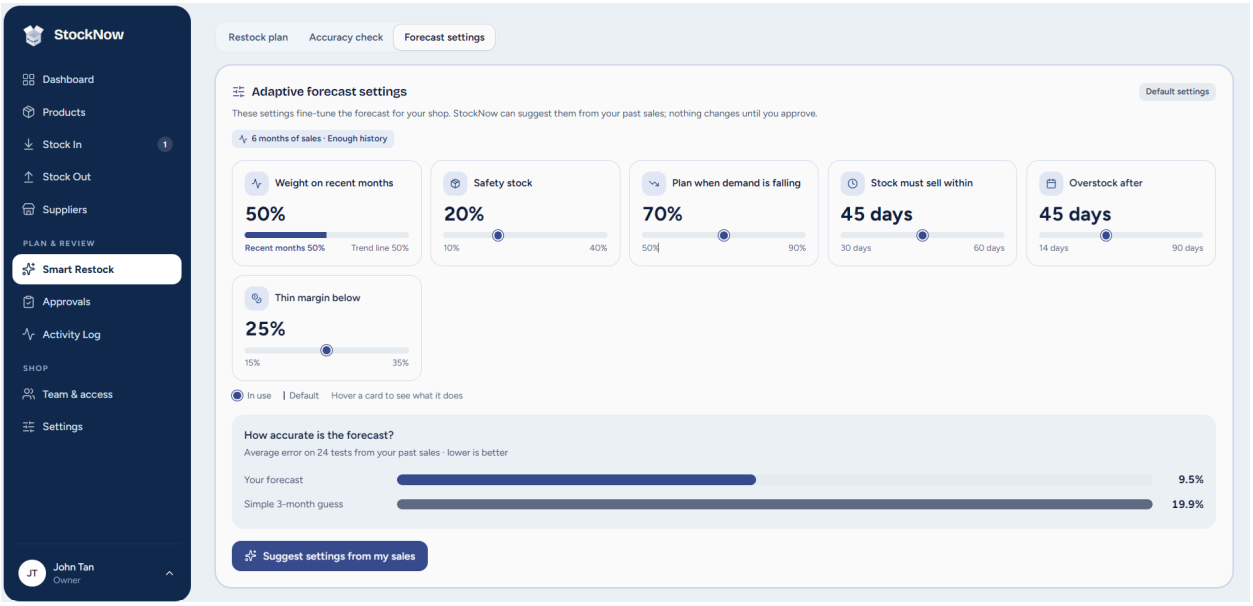
B1. Owner dashboard: today's numbers, budget left, and a "To do today" list built from automatic triggers.



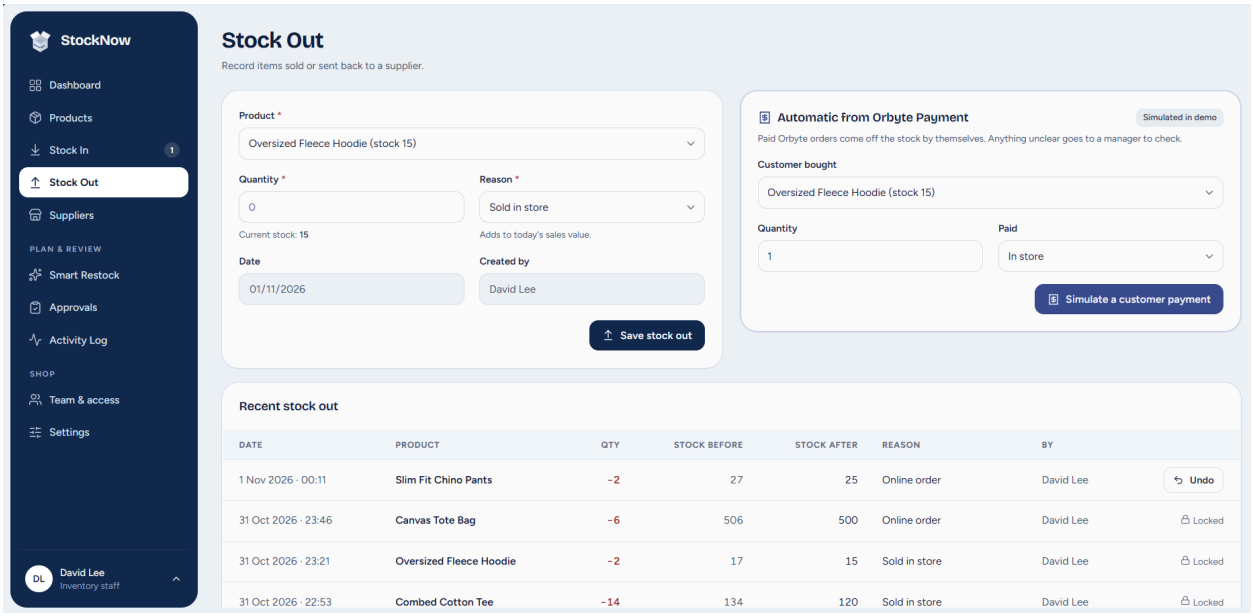
B2. Smart Restock: buy / wait / don't-buy, and a short summary written by Gemini from the engine's numbers (label "Written from these numbers"; the model name shows on hover).



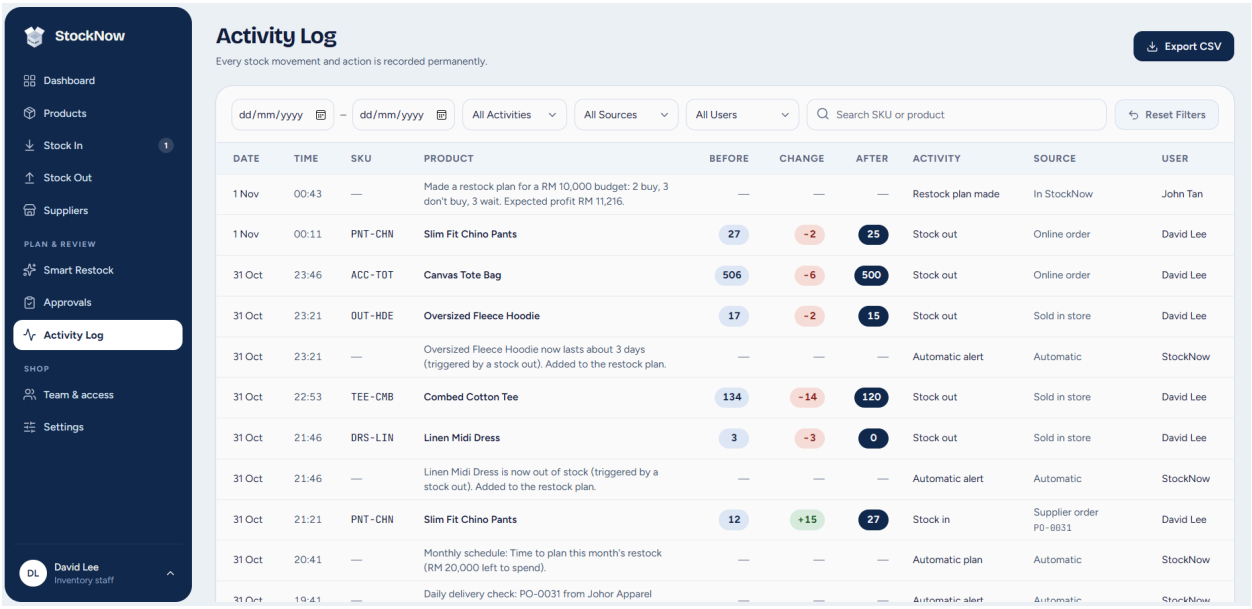
B3. "Show calculation": a chart of past sales, the trend line and next month's forecast, then every step of the maths.



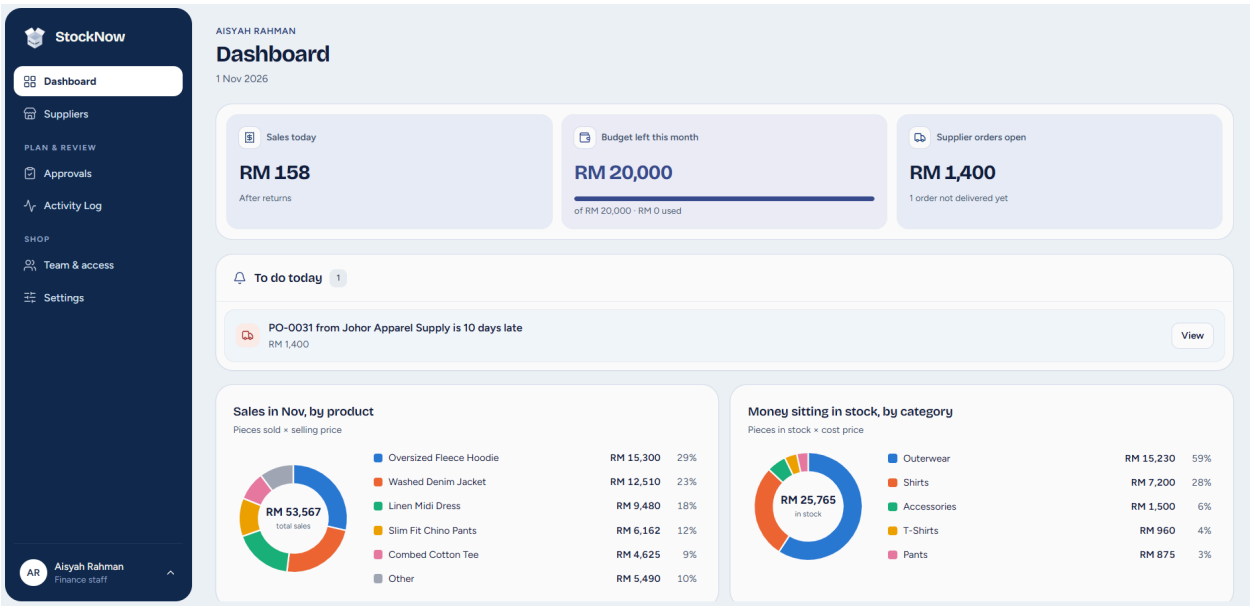
B4. Forecast settings in % and days: the suggestion from the shop's own sales (Gemini), safe ranges, and the backtest (9.5% → 5.1%, simple rule 19.9%) before the owner approves.



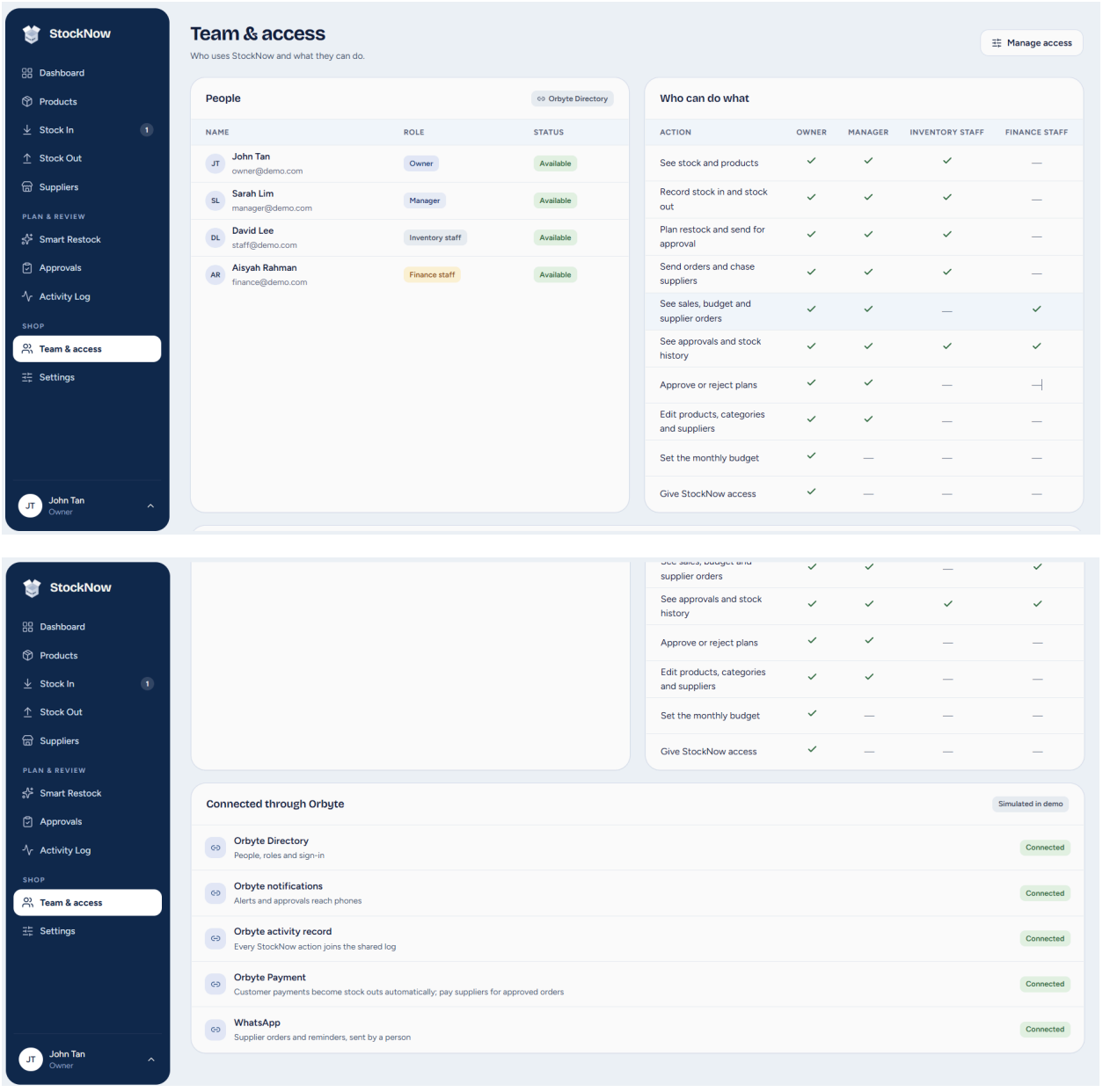
B5. Stock Out: a paid Orbyte order recorded automatically; an unknown item is held for a person (assumed interface, simulated in demo).



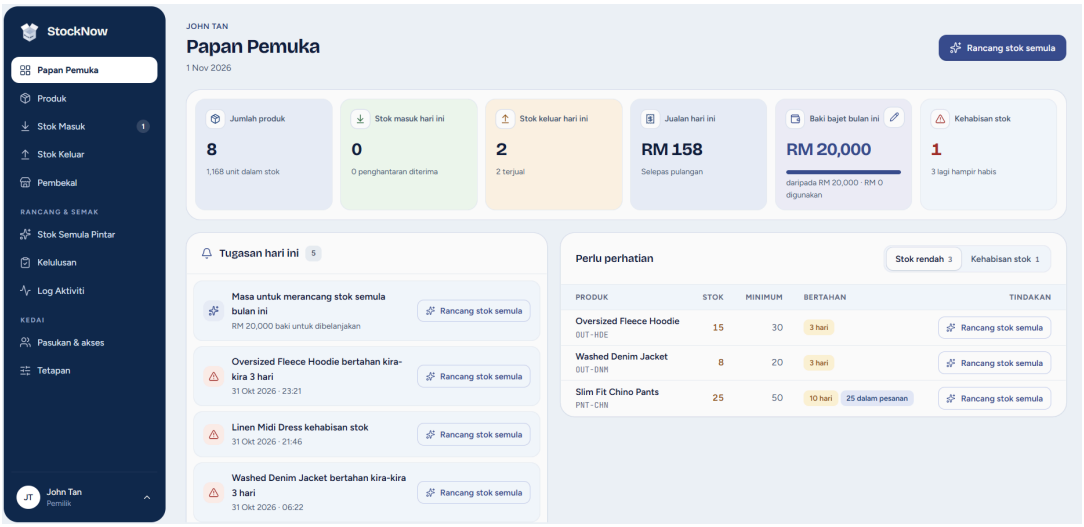
B6. Activity Log: every stock movement with before / change / after, source and user, plus approvals and settings; filters and CSV export.



B7. Finance staff view: only money pages, with sales by product and money sitting in stock by category.



B8. Team & access: Owner, Manager, Inventory staff and Finance staff, each with their own permissions and access connected through Orbyte.







B9. The same dashboard in Bahasa Melayu, Tamil, and Chinese.